

SCRAPING Y USO DE API

En este ejercicio se aprenderá a descargar datos de fuentes públicas en la web (por ejemplo Kaggle), hacer scraping de los mismos incluyéndolos en estructuras internas. Por último se utilizará una API de ejemplo para obtener información.

- Generar un archivo tipo Notebook de Python que contenga el código fuente de varios ejercicios aplicados a los conceptos vistos:

- Importación de sitio web con requests e impresión del contenido (cualquier sitio)
- Uso de librería BeautifulSoup y requests. Parseo de información (e.g. mercadolibre, despegar.com, etc.)
- Generación de un listado simple, ordenado, de los ítems que aparecen en las listas
- Uso de librería requests con respuestas de API.

```
#Importacion de sitio web con reques e impresion del contenido
import requests
from bs4 import BeautifulSoup
from pprint import pprint

ml = 'https://listado.mercadolibre.com.mx/twisted-wonderland_
data_ml = requests.get(ml)

#Impresion de la informacion del sitio web
pprint(data_ml.text)
```

✓ 0.4s

Python

```
('<html>\n'
 '  <head><meta http-equiv="Content-Type" content="text/html; ch
 '    <link rel="stylesheet" type="text/css"
 'href="https://http2.mlstatic.com/ui/navigation/2.3.5/mercadoli
 '  <style>\n'
 '    .ui-empty-state {\n'
 '      position: relative;\n'
 '      min-height: 25em\n'
 '    }\n'
 '\n')
```

Para realizar esta práctica lo que se hizo fue importar principalmente los módulos de *Requests*, *BeautifulSoup* y *PrettyPrint*. Permitiendo que la primera instrucción se realice utilizando el sitio web de Mercado Libre solo para extraer su estructura en HTML.

A partir de este punto se utilizará *prettyprint* para una mejor visualización

```
#Se separan los headers para una mejor visualizacion del pars
headers = {
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Methods': 'GET',
    'Access-Contro-Allow-Headers': 'Content-Type',
    'Access-Control-Max-Age': '3600',
    'user-Agent': 'Mozilla/5.0(X11); Ubuntu; Linux x36_64; rv
}
```

```
#Se realiza el request sobre el sitio de Scrape this page
sc = 'https://www.scrapethissite.com/pages/simple/'
data_sc = requests.get(sc, headers = headers)
```

En el segundo paso, la segunda instrucción se solicita hacer *parseo* de alguna página web por lo que se realizó principalmente es crear headers para crear un acceso a dicha página, como siguiente se almaceno la página dentro de una variable.

```
#Variable para realizar parseo y guardar la información
url_sc = []

html = BeautifulSoup(data_sc.text,'html.parser')
countries = html.select('.country')

#Se separan los paises, por nombre, capital, poblacion y area
for country in countries:
    name = country.select('.country-name')[0].get_text(strip)
    capital = country.select('.country-capital')[0].get_text(strip)
    population = country.select('.country-population')[0].get_text(strip)
    area = country.select('.country-area')[0].get_text(strip)
    url_sc.append({'name':name,'capital':capital,'population':population})

pprint(url_sc)
```

Como segunda parte de la segunda instrucción lo que se hizo fue tener una lista vacía donde se almacenaran los objetos extraídos con información de la página.

Siendo esta información de países, con su respectivo nombre, población, capital y área.

No se pudo realizar hacia páginas de ecommerce debido a que estas han actualizado su estructura en html, términos y condiciones, así como el uso de bots

```
#Lista de todos los items dentro de la pagina
page = requests.get(sc,headers)
soup = BeautifulSoup(data_sc.content, 'html.parser')

class_list = set()
tags = {tag.name for tag in soup.find_all()}

#Se iteran todos los tags
for tag in tags:
    #Encontrar todos los elementos del tag:
    for i in soup.find_all(tag):
        if i.has_attr('class'):
            if len(i['class']) !=0:
                class_list.add('.'.join(i['class']))

pprint(class_list)
```

En esta tercera instrucción se solicitó que se hiciera una lista ordenada de los ítems que aparecen en dicha pagina utilizada. Todos los tags se rectificaron y pertenecían a cada parte de lo que se extrajo de información

```
✓ 0.6s
{'col-md-12',
 'col-md-12text-center', 'col-md-12text-muted',
 'col-md-4country',
 'col-md-6',
```

```
#Obtencion de informacion desde una API
response_API = requests.get('https://makeup-api.herokuapp.com')
response_API.status_code
```

✓ 0.7s

Python

200

```
pprint(response_API.json())
```

✓ 0.0s

Python

```
[{'api_featured_image': 'https://s3.amazonaws.com/donovanbailey/production/images/maybelline/face-studio-master-hi-light-light-bronzer-formula-has-an-expert-balance-of-shade-shimmer-illuminator-for-natural-skin-goes.jpg',
  'brand': 'maybelline',
  'category': None,
  'created_at': '2016-10-01T18:36:15.012Z',
  'currency': None,
  'description': 'Maybelline Face Studio Master Hi-Light Light B',
  'price': '10.29',
  'price sign': None,
  'title': 'Maybelline Face Studio Master Hi-Light Light B',
  'url': 'https://www.maybelline.com/products/face-studio-master-hi-light-light-bronzer-formula-has-an-expert-balance-of-shade-shimmer-illuminator-for-natural-skin-goes'}]
```

En la última instrucción se solicita el uso de API, por lo que se inicio utilizando una de las API gratis y libres para uso practico como este. Que este caso se obtuvo sobre una línea de maquillaje, la marca a usar fue *Maybelline*

```
product_sample = response_API.json()[30]
```

✓ 0.0s

Python

```
product_sample
```

✓ 0.0s

Python

```
{'id': 177,
  'brand': 'maybelline',
  'name': 'Maybelline Line Stiletto Ultimate Precision Liquid Eye',
  'price': '10.29',
  'price sign': None,
  'title': 'Maybelline Line Stiletto Ultimate Precision Liquid Eye',
  'url': 'https://www.maybelline.com/products/line-stiletto-ultimate-precision-liquid-eye'}]
```

```
threereponse_API = requests.get('https://house-stock-watcher-')
threereponse_API.status_code
```

✓ 0.2s

Python

d6c6c
stile

403

a0

```
tworesponde_API = requests.get('https://data.police.uk/api/cr')
tworesponde_API.status_code
```

✓ 0.4s

Python

404

```
onerresponse_API = requests.get('https://list.ly/api/v4/meta?')
onerresponse_API.status_code
```

✓ 0.2s

Python

503

Se extrae un ejemplo del json proporcionado a lo que al final se comprueba si este se extrajo correctamente

Otras pruebas que se realizaron fue obtener distintas respuestas por parte de las API compartidas

Dando como resultados

403 – Prohibido el acceso

404 – No existe la pagina

503 – Error de servidor